

mdlout Technical Manual

James Clements III

mdlout project

2026-05-21

Abstract

This manual is the reference companion to the mdlout Markdown-to-Lout converter. It covers installation from the upstream Git repository, a ten-minute quick-start, the YAML frontmatter knobs that drive page geometry and document type, the full command-line interface, the internal Python API exposed by `mdlout.py`, and a troubleshooting appendix that catalogues the most common failure modes observed in the regression suite. The manual was itself typeset with mdlout, so any inconsistency between this text and the program's actual behaviour is by definition a bug.

mdlout Technical Manual

James Clements III

mdlout project

2026-05-21

Contents

1. Installation	1
1.1. Prerequisites	1
1.2. Cloning the repository	1
1.3. Building Lout	2
2. Quick start	2
2.1. A worked example	2
3. Configuration	3
3.1. Frontmatter keys	3
3.2. A complete frontmatter example {#sec:fm-example}	3
4. Command-line interface {#sec:cli}	4
4.1. Synopsis	4
4.2. Flag reference	4
4.3. Exit codes	4
5. Python API	5
5.1. Top-level functions	5
5.2. A worked C-extension example	5
6. Troubleshooting	6
6.1. "lout: file X not found"	6
6.2. "unresolved cross reference: Z"	7
6.3. "syntax error in symbol Y"	7
6.4. Blank or truncated output	7
7. Architecture	7
7.1. The three layers	7
7.2. The Python front end	8
7.3. The C middle layer	8
7.4. The thin glue layer	9
8. Regression testing	9
8.1. Snippet diffs	9
8.2. Example builds	9
8.3. Browser tests	9
9. Continuous integration	10
9.1. Workflow stages	10

9.2. Caching	11
10. Security considerations	11
11. Performance tuning	11
11.1. Wall-clock figures	12
11.2. HTML output size	12
12. Frequently asked questions	12
12.1. Why Lout and not LaTeX?	12
12.2. Why a Python front end?	12
12.3. Why not Pandoc?	13
12.4. Can mdlout produce ePub or DOCX?	13
12.5. What about right-to-left text?	13
12.6. Why is the SVG output so much larger than the PostScript?	13
13. Glossary	13
14. Appendix A: Configuration reference	14
15. Appendix B: A complete worked example	15
16. Introduction	15
17. Appendix C: Recipes	16
17.1. Recipe: A research paper	16
17.2. Recipe: A long-form book chapter	16
17.3. Recipe: A formal letter	17
17.4. Recipe: A two-column CV	17
17.5. Recipe: A slide deck	18
18. Appendix D: Implementation notes	18
18.1. Why a single-file Python script?	18
18.2. Why a flat list of Block dataclasses?	18
18.3. Why three Lout passes?	19
18.4. Why ASCII PostScript output?	19
18.5. Why no caching?	19
18.6. Why URW++ Nimbus fonts?	19
19. Appendix E: Compatibility matrix	19
20. Appendix F: A worked migration case study	20
20.1. The source document	20
20.2. What translated cleanly	20
20.3. What did not translate	21
20.4. The total time budget	21
20.5. Lessons learned	22
21. References and further reading	22
22. References	22

1. Installation

mdlout is a single-file Python 3.10+ script with no third-party Python dependencies. The runtime requirements are a built copy of Lout (the C document formatting system, version 3.43) and – only for the PDF pipeline – Ghostscript's 'ps2pdf'.

1.1. Prerequisites

The following tools must be on your '\$PATH' (or in a location mdlout can find via its built-in discovery):

Tool	Required for	Minimum version
'python3'	the converter	3.10
'lout'	both pipelines	3.43 (the fork)
'ps2pdf'	PDF mode only	Ghostscript 9.50
'rsvg-convert'	SVG-image PDF	librsvg 2.50
'chromium'	browser tests	90

Everything except Python and Lout is optional in the strict sense; the HTML path runs happily without 'ps2pdf' or 'rsvg-convert' as long as no Markdown image references a .svg file.

1.2. Cloning the repository

The Lout C implementation lives in a Git submodule next to the Python front end. The two are versioned together so that any change to the SVG back end is co-located with the converter that drives it:

```
git clone https://github.com/jclements3/mdlout.git
cd mdlout
git submodule update --init
cd lout && git checkout svg-backend && cd ..
```

The 'svg-backend' branch is the maintained working branch; the default branch in the submodule does **not** contain 'z53.c' and will fail to build the SVG pipeline.

1.3. Building Lout

The Lout binary is built with a stock 'make' invocation. The fork uses a flat source layout (no src/ subdirs):

```
cd lout
make all
```

The build produces two binaries directly in 'lout/:lout' itself and the auxiliary 'prg2lout'. mdlout finds them automatically; you do not need to make install.

Reproducible builds

The makefile defaults to ‘CHARIN=1 CHAROUT=0’ – ISO-Latin-1 input, 7-bit ASCII PostScript output. Both regression suites assume those defaults; override them only if you know what you are doing.

2. Quick start

The smallest possible mdlout invocation reads a Markdown file and writes the output file next to it. The two invocations below cover both formats:

```
# HTML mode (default): writes hello.html next to hello.md
./mdlout.py hello.md

# PDF mode: writes hello.pdf next to hello.md
./mdlout.py hello.md --format=pdf
```

If ‘hello.md’ contains the single line `# Hello, world.` then the HTML output is a self-contained file with the URW++ Nimbus web fonts inlined, the SVG produced by Lout’s `sz53.c` back end embedded inline, and a brief stylesheet. Open it in any browser.

The PDF pipeline goes through Lout’s frozen PostScript back end (`z49.c`) and Ghostscript’s `sps2pdf` wrapper. See section @sec:cli for the full flag reference.

2.1. A worked example

A more representative input demonstrates frontmatter, sections, code, and math. Save the following as ‘hello.md’:

```
---
type: report
title: My Report
author: Jane Doe
---

# Introduction

A short paragraph mentioning some math.

(Python code goes here.)
```

Build it twice – once per format – and inspect both outputs in parallel. The two pipelines share frontmatter parsing, block-level parsing, and Lout source generation; only the final back end differs.

3. Configuration

All persistent configuration lives in the YAML frontmatter block at the top of each `.md` file. There is no global config file; mdlout is intentionally context-free.

3.1. Frontmatter keys

The most commonly used keys, grouped by what they affect:

Document type. `type:selects one of doc(default), report, book, or slides`. Each maps to a different Lout package and therefore to a different page template.

Page geometry. `page:accepts A4, A5, Letter, etc.; orientation:accepts Portrait or Landscape; columns:accepts an integer column count; the four*-margin:keys take Lout length expressions (2.5c, 1.0in)`.

Typography. `font:sets the running text font and size (Times Base 11p is the default); para-gap:and para-indent:control paragraph spacing.`

Cross-references. `contents: Yes adds a table of contents; section-numbers: Arabic(or Roman, None) drives the numbering scheme; cover: Yes adds a cover sheet to reports.`

3.2. A complete frontmatter example {#sec:fm-example}

```
---
type: report
title: A Comparative Study
author: J. L. Clements
date: 2026-05-21
cover: Yes
contents: Yes
page: A4
columns: 2
section-numbers: Arabic
para-indent: 0f
para-gap: 1.0v
font: Times Base 11p
---
```

The block above is the canonical research-paper frontmatter. Refer back to it whenever a section in the rest of this manual asks for "a report frontmatter".

4. Command-line interface {#sec:cli}

mdlout's CLI is intentionally small. The most common invocations need only one or two flags.

4.1. Synopsis

```
# Build an HTML output (default format):  
./mdlout.py INPUT [OPTIONS]  
  
# Build a PDF output:  
./mdlout.py INPUT --format=pdf [OPTIONS]
```

4.2. Flag reference

The full set of supported flags:

Flag	Default	Effect
'-format'	'html'	Pick 'htmlorpdf'
'-o, -output'	derived	Output file path
'-lout-only'	off	Stop after the Markdown-to-Lout phase
'-ps'	off	Stop at the PostScript stage
'-external-assets'	off	Load KaTeX / abcjsharp / hljs from CDN
'-no-math-engine'	off	Omit KaTeX entirely
'-no-music-engine'	off	Omit abcjsharp entirely
'-no-highlight'	off	Omit highlight.js
'-watch'	off	Rebuild on every source save
'-serve [PORT]'	8080	HTTP server with SSE live-reload
'-mydefs'	auto	Path to a custom 'mydefs' file
'-lout-args'	none	Extra args to pass to the Lout binary

The '-watchand-serve' flags are documented in more detail in the user's guide; here we focus on the build-once pipeline. The "derived" default for -o is the input basename with the appropriate extension swapped in (.md->.htmlor.pdf).

4.3. Exit codes

mdlout follows the usual Unix conventions:

Code	Meaning
0	Build succeeded

1	indent ctr	Build failed (any phase)
2	indent ctr	Invalid command-line arguments
77	indent ctr	Optional dependency missing, build skipped

Code 77 is the autotools "skipped" convention; CI systems are expected to treat it as a soft skip rather than a hard failure.^[1]

harness. mdlout adopts it so that an unconfigured CI runner does not spuriously flag an entire build as broken.

5. Python API

mdlout's surface area is the script's `main()` function and its CLI flags; the internal symbols are not a stable API. Even so, a handful of them are useful enough to mention by name for people writing tools that wrap mdlout (for example, a static-site generator that wants to pre-process Markdown before mdlout sees it).

5.1. Top-level functions

The interesting entry points, each one a top-level `def` in `mdlout.py`:

```
def parse_frontmatter(text: str) -> tuple[dict, str]:
    """Split YAML frontmatter from the markdown body."""

def parse_markdown(text: str) -> list[Block]:
    """Block-level Markdown parser; returns a list of Block records."""

def convert_inline(text: str) -> str:
    """Inline-level Markdown -> Lout source. Idempotent on plain text."""

def generate_lout(blocks, frontmatter=None) -> str:
    """Top-level: list of Block + frontmatter dict -> Lout source string."""
```

The `Block` dataclass and the `BlockType` enum are also stable in the sense that the regression suite would catch breaking changes.

5.2. A worked C-extension example

For users who want to embed mdlout in a C program, the simplest approach is to invoke `mdlout.py` as a subprocess and consume its output. A minimal driver:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

¹The convention dates back to GNU automake's testsuite

```
int build_html(const char *md_path, const char *out_path) {
    char cmd[1024];
    int n = snprintf(cmd, sizeof(cmd),
                     "./mdlout.py %s -o %s",
                     md_path, out_path);
    if (n <= 0 || (size_t) n >= sizeof(cmd)) return -1;
    return system(cmd);
}

int main(int argc, char **argv) {
    if (argc != 3) {
        fprintf(stderr, "usage: %s INPUT.md OUTPUT.html\n", argv[0]);
        return 2;
    }
    return build_html(argv[1], argv[2]);
}
```

The same approach in shell (one-liner, useful for CI):

```
for f in *.md; do
    ./mdlout.py "$f" -o "out/${f%.md}.html" || exit 1
done
```

6. Troubleshooting

The error messages below are the ones the regression suite saw most often during the SVG back-end bring-up. Each is paired with the fix that resolved it.

6.1. "lout: file X not found"

The most common cause is a missing submodule checkout: the Lout library files live under `lout/include/andlout/data/`, and those directories are empty until the submodule is initialised. The fix is the second half of the install recipe in @sec:fm-example above:

```
git submodule update --init
cd lout && git checkout svg-backend && cd ..
```

A subtler cause is a typo in a `@SysInclude { foo }` line in your `ownmydefs` file. Lout reports the literal name it tried to find, so the typo is usually visible in the error message itself.

6.2. "unresolved cross reference: Z"

Lout normally needs three passes to resolve forward references (the first pass discovers labels, the second resolves them, the third typesets the resolved text). `mdlout` runs Lout up to three times by default; if you see this warning **after** the third pass then the target really is missing. Search the document for the label and check for spelling drift.

Forward-reference fragility

If you wrap an entire document in a single `@OneRow { ... }` block, Lout's forward-reference resolver can sometimes get stuck in a fixed-point loop where the page count keeps changing on each pass. The workaround is to break the document into smaller chunks that resolve independently.

6.3. "syntax error in symbol Y"

Almost always an unbalanced `{or}` inside a `raw` Lout fence. The fix is to count braces in the offending fence; `mdlout` does not do brace-balancing on `raw` Lout fence content because doing so would interfere with intentionally unbalanced macros like `@Endinside@Verbatim`.

6.4. Blank or truncated output

Frequently a missing `mydefsfile` when a `raw` Lout fence references a custom macro. Pass `mydefs path/to/file` or put the file next to the `input.md`; `mdlout` copies it into the build directory before invoking Lout.

7. Architecture

This section dissects the toolchain's internal layering for the benefit of contributors and the morbidly curious. `mdlout` sits on top of an unusual stack – a Python front end driving a C document formatting system – and the design choices behind that stack are worth surfacing.

7.1. The three layers

`mdlout` is a three-layer pipeline. The top layer is a Python script that converts Markdown to Lout source; the middle layer is the Lout binary itself, written in ANSI C, that converts Lout source to either PostScript or SVG; the bottom layer is a thin glue script that wraps the SVG in HTML or feeds the PostScript through `ps2pdf`¹ to make a PDF.[2]

first prototype of `mdlout` invoked LaTeX rather than Lout, on the theory that LaTeX's larger user base would make for easier debugging. That plan fell apart on day three, when a single typo in a math block produced a fifteen-page error trace that bore no obvious relationship to the input. Lout's error messages are not always pleasant, but they are reliably *small*.

The pipeline is one-way: Markdown flows in, Lout flows through the middle, output emerges at the bottom. There is no back-pressure, no caching, and no incremental rebuild logic. A full build of the regression corpus – twenty-six snippets and a dozen example documents – takes about ninety seconds on a modern laptop. That is fast enough that incremental builds would be a complication for little gain.

7.2. The Python front end

The front end is a single file of approximately three thousand lines of Python. It has four phases, each implemented as a top-level function:

¹This layering was not the original plan. The

Phase	Function
Frontmatter parse	'parse_front
Block-level parse	'parse_mark
Inline conversion	'convert_in
Lout generation	'generate_l

The phases are independent: each one consumes the previous phase's output as a self-contained data structure. That independence is what makes mdlout testable. The unit tests in `'tests/` exercise each phase separately and the regression suite verifies the full pipeline end to end.

The inline-conversion phase is the most subtle. Lout's metacharacters (`@`, `{`, `}`, `"`, `\`) overlap with several Markdown constructs (links, code spans, math), so naive escaping turns valid Markdown into syntactically broken Lout. mdlout addresses this by running a placeholder pass first: every protected span (code, link target, math expression, raw Lout fence) is replaced with a unique sentinel string, the surrounding plain text is then escaped, and finally the sentinels are restored to their original content. The placeholder strings are chosen to be guaranteed unique across the document; the restore pass is sorted stable, which matters when one sentinel appears as a literal substring of another.^[3]¹

shorter sentinels that collided occasionally with user content, with predictably entertaining results. The current implementation uses forty-character UUID-like strings with a fixed prefix and a numeric counter – collisions are now theoretically impossible.

7.3. The C middle layer

The Lout binary is a port of Jeffrey Kingston's original 1993 implementation, maintained as a fork at github.com/william8000/lout. The mdlout project carries its own further fork on the branch `'svg-back-end`. The fork adds a new output back end (`z53.c`, the SVG emitter) while leaving all the other back ends – PlainText, PostScript, PDF – frozen and bit-identical to the pre-fork era.

The SVG back end is a sibling of the PostScript back end, not a replacement. It implements the same `'BACK_ENDinterface defined inexterns.h` – text emission, graphic state save/restore, coordinate transforms, link generation, image inclusion – and uses Lout's existing galley engine, font-metrics service, and colour service. Only the emission layer differs: where the PostScript back end writes PostScript drawing commands, the SVG back end writes SVG elements.

The choice to fork rather than to write a new converter was a hard one. A clean rewrite would have produced a cleaner result but at the cost of losing the rich set of features Lout already implements – hyphenation, paragraph filling, footnote floats, cross-references, language-specific quoting, the equation typesetter, the diagram package. A wrapper around an existing PDF or HTML library would have been simpler but would have abandoned the typographic quality that makes Lout interesting in the first place. The fork was the compromise that preserved the things that work while making room for the SVG output that did not previously exist.

7.4. The thin glue layer

The bottom of the stack is the smallest piece of code in the project: roughly two hundred lines of Python that wrap the SVG output in HTML (adding KaTeX, abcjsharp, and highlight.js for browser-side rendering of math, music, and code) or run the PostScript output through `'ps2pdf` (adding `nothing`) .

¹An early version of the placeholder system used

The HTML wrapper is the more interesting half because it has to make choices about which assets to inline and which to load from CDN. Those choices are exposed as the `external-assets` flag.

8. Regression testing

mdlout's regression suite has three layers: snippet-level visual diffs that compare the PostScript and SVG back ends on small inputs, example-document builds that exercise the full pipeline on realistic inputs, and browser tests that confirm the HTML output actually renders in a real Chromium.

8.1. Snippet diffs

The smallest tests are individual `.ltx` files (Lout source) under `tests/snippets/`. Each snippet is rendered twice – once to PostScript and once to SVG – and the two renderings are rasterised to PNG at 150 dpi. The PNGs are compared with ImageMagick's metric AE and a structural-similarity metric (`compute_ssim`). A snippet passes when both the absolute pixel difference and the SSIM score sit inside thresholds appropriate to its content: 5% for text-only snippets, 20% for graphics-heavy ones.

The thresholds were chosen empirically. Text-only snippets *should* pixel-match between the two back ends, because both ultimately rely on the same URW++ Nimbus font metrics; the 5% tolerance exists only to absorb the unavoidable rasteriser jitter from rendering type at limited pixel resolution. Graphics-heavy snippets – `@Diag` trees, shape macros, fills – have higher unavoidable variance because the two back ends use different stroke models, so the 20% threshold is a deliberate looseness.^[4]

(e.g. "ninety-fifth percentile of pixel differences must be below N") instead of a fixed absolute number, but the small sample size per snippet (one image) makes percentile analysis statistically noisy. The fixed threshold is conservative but stable.

8.2. Example builds

The middle layer of testing builds every `examples/*.mdfile` in both `format=html` and `format=pdf` modes and confirms each build returns exit code zero. This is the most direct verification that mdlout end-to-end works on realistic input. The committed reference outputs under `examples/out/` exist for visual inspection, but the `build-doesn't-fail` test is what the CI gate checks. A failed example build halts the build process; a visually regressed example produces a noticeable change in the gallery but does not by itself fail CI.

8.3. Browser tests

The third layer is `tests/browser_test.py`, which uses headless Chromium to render every example HTML file and checks that the post-JavaScript DOM contains the elements it should. The checks are intentionally narrow: every `<span class="math">` should have become a `katex` tree, every `<div class="abc-music">` should contain at least one `<svg>`, every `<code class="language-X">` should have at least one `hljs-*` token span, every `href="#x"` should have a matching `id="x"` somewhere in the document. The test does not try to verify visual fidelity – that is what the snippet diffs are for. It only verifies that the *features* the HTML

¹We considered using a percentile-based threshold

claims to use are wired up correctly.

9. Continuous integration

mdlout's CI runs the three test layers in sequence, on every push to the main branch and on every pull request. The CI configuration is a single GitHub Actions workflow file at `github/workflows/ci.yml` in the repository.

9.1. Workflow stages

The CI workflow has four stages, each of which must pass before the next begins:

```
jobs:
  build:
    runs-on: ubuntu-22.04
    steps:
      - uses: actions/checkout@v4
        with:
          submodules: true
      - name: Install build deps
        run: |
          sudo apt-get update
          sudo apt-get install -y gcc make ghostscript librsvg2-bin chromium
      - name: Build Lout
        run: cd lout && make all
      - name: Run snippet diffs
        run: bash tests/run_all.sh
      - name: Build all examples
        run: |
          for f in examples/*.md; do
            ./mdlout.py "$f" -o "/tmp/${basename "${f%.md}.html"}" || exit 1
            ./mdlout.py "$f" --format=pdf -o "/tmp/${basename "${f%.md}.pdf"}" || exit 1
          done
      - name: Run browser tests
        run: bash tests/browser_test.sh
```

The four stages – build Lout, snippet diffs, example builds, browser tests – take about three minutes of wall-clock time on a standard GitHub Actions runner. The largest single phase is the Lout build, which compiles fifty-three C source files; the next largest is the browser-test phase, which spawns one headless Chromium per HTML example.

9.2. Caching

The Lout build is cacheable: nothing in the C sources depends on the running OS version or the Python toolchain, so a previously-built Lout binary remains valid as long as the submodule SHA has not changed. The CI workflow keys the cache on the submodule SHA and restores it on every run; a clean build of Lout is only needed when the submodule advances. This shaves about a minute off the typical PR build.

10. Security considerations

mdlout is a build tool, not a runtime, but it still has a small security surface. Worth noting:

- **Untrusted Markdown input.** mdlout's parser is hand-written and does not invoke `eval`, `exec`, or any other dynamic-code path. Frontmatter is parsed as YAML scalar key/value pairs only – there is no support for YAML anchors, references, or tags, all of which can carry RCE payloads in a permissive parser.
- **Raw Lout fences.** A `““lout“fenced block is passed through verbatim to the Lout binary. If the Markdown input is untrusted, the raw Lout fence is an obvious vector for a malicious@SysInclude { /etc/passwd }. mdlout does not sandbox this today; deployments that ingest untrusted input should either rejectlout‘ fences at the gate or sandbox the Lout binary separately.`
- **Image inclusion.** Markdown image references can name a local filesystem path, which mdlout reads at build time. If the input is untrusted, a malicious image reference can ex-filtrate file contents into the output. The mitigation is the same: gate Markdown image refs against a known-good directory, or sandbox the build process.
- **HTML output.** The HTML output inlines KaTeX, abcjsharp, and highlight.js. All three are MIT-licensed and reasonably well-audited; mdlout pins them to specific known-good versions (KaTeX 0.16.10, abcjsharp commit harp-grandstaff, highlight.js 11.9.0) and emits Subresource-Integrity hashes when using `‘external-assets‘`.

RCE via raw Lout fence

The raw Lout fence is a deliberate escape hatch. Treat any workflow that ingests user-submitted Markdown as if it were arbitrary code-execution: either strip `‘lout‘ fences before invoking mdlout or run mdlout inside a sandbox that has no network or filesystem access beyond the build directory.`

11. Performance tuning

mdlout is fast enough out of the box for documents up to a few hundred pages. The numbers below give a rough sense of the time budget; they are not benchmarks in any rigorous sense, only ballpark figures gathered from the regression suite running on a 2023-vintage laptop.[5]¹

been done. The figures here come from informal timing of the regression suite. Take them as orders of magnitude, not as contractual guarantees.

11.1. Wall-clock figures

A representative breakdown of wall-clock time for a single `‘./mdlout.py input.md -format=pdf‘ invocation, on an input document of about a hundred pages:`

¹Proper benchmarking is on the wish list but has not

Phase	Time	
Markdown parsing	60 ms	indent
Lout source generation	40 ms	indent
Lout invocation (3 passes)	1700 ms	inde
'ps2pdf'	320 ms	inden

The Lout invocation dominates by an order of magnitude. The three-pass structure is what allows Lout to resolve forward cross-references, but it does mean that even a tiny document pays for three full typesetting passes. Documents that have no cross-references (`'[TOC], @fig:label, [@cite]'`) can in principle be built with a single pass; mdlout does not currently do this optimisation.

11.2. HTML output size

The default HTML output inlines all the runtime assets: URW++ Nimbus fonts (several MB), KaTeX (about 280 kB), abcjsharp (about 850 kB), highlight.js (about 130 kB), and the SVG content itself. A hundred-page document with light math, no music, and no syntax highlighting produces an HTML file of roughly three megabytes; turn on music and the figure climbs to about four. Use `'-external-assets'` to push the JS/CSS dependencies to a CDN; that shaves the per-file size to about a third without changing the rendered result.

The `'-no-math-engine, -no-music-engine, and-no-highlight'` flags drop the corresponding library entirely. Use them when you know the document does not need the feature – saves about 1.5 MB in the typical case.

12. Frequently asked questions

12.1. Why Lout and not LaTeX?

LaTeX has a much larger user base and a more comprehensive ecosystem, but its error messages are unusually difficult to parse for the non-expert and its compilation model interleaves macro expansion with typesetting in ways that make automated tooling fragile. Lout, by contrast, has a clean separation between its definition reader and its typesetting engine, which makes it well-suited to driving from a front end like mdlout. The Lout community is small but well-defined, and the language itself is small enough to keep in one's head.

12.2. Why a Python front end?

Python was chosen for ubiquity: every Unix system has Python 3 installed by default, and the standard library is rich enough that mdlout has no third-party dependencies. A Go or Rust front end would have been smaller and faster but would have added a build step that non-developer users would not appreciate.

12.3. Why not Pandoc?

Pandoc is a remarkable tool and covers many of the same use cases as mdlout, but it does not target Lout. Adding Lout as a Pandoc output format would have been a much larger project than writing mdlout from scratch, because Pandoc's internal AST does not have a natural mapping to Lout's galley model.

12.4. Can mdlout produce ePub or DOCX?

Not directly. mdlout's pipeline goes Markdown to Lout, and Lout's output back ends are PostScript, PDF, SVG, and plain text. ePub and DOCX would need their own back ends, which is a substantial piece of C engineering. The path of least resistance is to convert mdlout's HTML output to ePub via Pandoc (`'pandoc input.html -o output.epub'`) and to DOCX via LibreOffice (`'soffice -headless -convert-to docx input.html'`).

12.5. What about right-to-left text?

Lout has limited RTL support: it can typeset Hebrew and Arabic with the right font, but the line-breaking algorithm is left-to-right by default. mdlout does not currently expose any RTL knobs in frontmatter; users who need RTL text should drop into a raw-Lout fenced block.

12.6. Why is the SVG output so much larger than the PostScript?

The SVG back end emits one `<text>` element per word (or even per glyph, in text-as-paths mode), while the PostScript back end can use its own font-state machine to elide most positioning commands. Compression closes most of the gap – the inlined SVG-in-HTML is about three times the size of the equivalent PDF, mostly font CSS.

13. Glossary

This glossary collects the project-specific terminology used throughout the manual. Entries are alphabetical.

Back end. A Lout module (one of `'z48.c, z49.c, z51.c, orz53.c'`) that consumes the galley engine's typeset output and emits a particular output format (PDF, PostScript, plain text, SVG respectively).

Block. mdlout's internal representation of a Markdown block-level construct: heading, paragraph, list, table, code block, math block, raw-Lout fence, etc. Implemented as a Python dataclass.

Cross-reference. A label-and-reference pair in a document, used to refer to figures, tables, sections, footnotes, or bibliography entries by name rather than by hard-coded number. Lout's three-pass build cycle is what makes cross-references resolvable.

Frontmatter. A YAML block at the very top of a Markdown file, delimited by `'–'` lines, used to set document-wide configuration that does not naturally fit into the body. mdlout's frontmatter parser is intentionally small and supports only top-level scalar key/value pairs.

Galley. Lout's term for a stream of typeset material flowing through the document. The galley engine is what schedules text into pages, handles float placement, manages footnote columns, etc. mdlout never interacts with the galley engine directly; that is Lout's responsibility.

Placeholder. A unique sentinel string used by `'convert_inline'` to protect code, link targets, math expressions, and raw-Lout fences from being mangled by the surrounding inline-escape pass.

Snippet. A small `'ltfile undertests/snippets/'` used as the smallest unit of regression testing. Each snippet renders to both PostScript and SVG; the two outputs are pixel-compared.

SSIM. Structural Similarity Index, a perceptual image-comparison metric. Used by `'tests/compare.py'` alongside the absolute pixel difference (`-metric AE'`) to classify

regression-test outputs as pass-excellent or fail.

SVG back end. The new output module `'z53.c'`, added in the `svg-backend` branch of the maintainer's Lout fork. Selected by `lout -G`; emits one `<svg>` element per page.

14. Appendix A: Configuration reference

The following table catalogues every frontmatter key `mdlout` recognises, the default value, and the Lout option it maps to. Use this as a fast lookup when the body of the manual is too verbose.

Key	Default	Lout target
<code>'type'</code>	<code>'doc'</code>	(selects package)
<code>'title'</code>	(empty)	<code>'@Title'</code>
<code>'author'</code>	(empty)	<code>'@Author'</code>
<code>'institution'</code>	(empty)	<code>'@Institution'</code>
<code>'date'</code>	(empty)	<code>'@DateLine'</code>
<code>'font'</code>	Times Base 11p	<code>'@InitialFont'</code>
<code>'language'</code>	English	<code>'@InitialLanguage'</code>
<code>'colour'</code>	(empty)	<code>'@InitialColour'</code>
<code>'page'</code>	A4	<code>'@PageType'</code>
<code>'orientation'</code>	Portrait	<code>'@PageOrientation'</code>
<code>'top-margin'</code>	2.5c	<code>'@TopMargin'</code>
<code>'foot-margin'</code>	2.5c	<code>'@FootMargin'</code>
<code>'left-margin'</code>	2.5c	<code>'@OddLeftMargin'</code>
<code>'right-margin'</code>	2.5c	<code>'@OddRightMargin'</code>
<code>'columns'</code>	1	<code>'@ColumnNumber'</code>
<code>'column-gap'</code>	0.5c	<code>'@ColumnGap'</code>
<code>'page-headers'</code>	None	<code>'@PageHeaders'</code>
<code>'page-numbers'</code>	Arabic	<code>'@PageNumbers'</code>
<code>'contents'</code>	No	<code>'@MakeContents'</code>
<code>'index'</code>	No	<code>'@MakeIndex'</code>
<code>'para-gap'</code>	1.0v	<code>'@ParaGap'</code>
<code>'para-indent'</code>	2f	<code>'@ParaIndent'</code>
<code>'cover'</code>	No	<code>'@CoverSheet'</code>
<code>'section-numbers'</code>	Arabic	<code>'@SectionNumbers'</code>
<code>'chapter-numbers'</code>	Arabic	<code>'@ChapterNumbers'</code>
<code>'chapter-start'</code>	NewPage	<code>'@ChapterStartPages'</code>
<code>'heading-font'</code>	(font family)	<code>'@HeadingFont'</code>
<code>'fixed-font'</code>	Courier	<code>'@FixedWidthFont'</code>

'optimize-pages'	No	'@OptimizePages'
'references_format'	numeric	(mdlout-internal)
'abstract'	(empty)	'@Abstract'

Keys not in this table are silently ignored, which makes mdlout robust against frontmatter that targets some other downstream tool.

15. Appendix B: A complete worked example

The complete frontmatter and body of a minimal but realistic two-column report. The frontmatter:

```
---
type: report
title: A Worked Example
author: J. Doe
cover: Yes
contents: Yes
page: A4
columns: 2
section-numbers: Arabic
font: Times Base 11p
---
```

The body sketch (Markdown, untagged so it survives the fenced-block parser):

16. Introduction

A short paragraph mentioning $e^{i\pi} + 1 = 0$.

Method

See @sec:fm-example for the frontmatter.

A Python snippet would go here as a fenced code block tagged python.

This file builds in both formats:

```
# Build the HTML rendering:
./mdlout.py worked.md

# Build the PDF rendering:
./mdlout.py worked.md --format=pdf
```

producing 'worked.html' and 'worked.pdf' next to the source.

17. Appendix C: Recipes

This appendix collects worked examples for the document genres that mdlout users ask about most often. Each recipe is a complete, self-contained Markdown source ready to copy into a new file and build. The recipes are derived from real documents written with mdlout; the placeholder names and content are fictitious.

17.1. Recipe: A research paper

A two-column report with a cover sheet, abstract, table of contents, numbered sections, math, tables, and a numbered bibliography. Use the following frontmatter:

```
---
type: report
title: A Comparative Study of Newton-Cotes Rules
author: J. L. Clements
institution: mdlout project
date: 2026-05-20
cover: Yes
contents: Yes
page: A4
columns: 2
section-numbers: Arabic
font: Times Base 11p
abstract: |
    We revisit two classical Newton-Cotes quadrature rules and
    quantify their accuracy on a battery of test integrals.
---
```

The body should open with a '[TOC]placeholder, then numbered#sections for introduction, method, results, and references. Inline citations use the[@key]syntax and the bibliography lives in a# Referencessection with[@key]: ...'lines.

17.2. Recipe: A long-form book chapter

A single-column book chapter with running heads, generous margins, Roman chapter numerals, and a pull-quote in italics. Frontmatter:

```
---
type: book
title: The Cartographers of Veil
author: James Clements III
font: Times Base 11p
page: A5
top-margin: 2.0c
foot-margin: 2.0c
left-margin: 2.0c
right-margin: 2.0c
para-gap: 0b
para-indent: 2f
```

```
chapter-numbers: Roman
section-numbers: None
page-headers: Titles
---
```

In the body, each `#heading` becomes a chapter with Roman numbering. `##` becomes a section, `###` a subsection. Pull quotes are blockquotes wrapped with `>` markers; `mdlout` renders them as centred italic displays in the PDF.

17.3. Recipe: A formal letter

A US business letter on a single page of US Letter, no headers, with the sender's address right-aligned and the salutation and signature in raw-Lout passthrough. Frontmatter:

```
---
type: doc
font: Times Base 11p
page: Letter
top-margin: 2.5c
foot-margin: 2.5c
left-margin: 2.5c
right-margin: 2.5c
para-gap: 1.2v
para-indent: 0f
page-headers: None
---
```

The body uses a sequence of raw-Lout fenced blocks for the sender address, the date, the recipient address, and the signature. The body of the letter sits between those blocks as regular Markdown paragraphs. See `'examples/letter.md'` for a complete worked template.

17.4. Recipe: A two-column CV

A two-column CV using `'type: docwithcolumns: 2'`, the candidate's name as a centred banner, and `a@TaggedList` of skills. Frontmatter:

```
---
type: doc
font: Times Base 10p
page: Letter
columns: 2
column-gap: 0.8c
para-gap: 0.6v
para-indent: 0f
page-headers: None
---
```

In the body, use a single raw-Lout block at the top for the `'@CentredDisplaybanner'`. The body sections (`## Summary`, `## Education`, `## Experience`) flow into the two

columns;@TaggedListblocks are well suited to the skills section. Seeexamples/cv.md' for a complete worked template.

17.5. Recipe: A slide deck

A talk in slide form. One '#heading per slide. Keep code, tables, and@Diag'out of slides for now. Frontmatter:

```
---
type: slides
title: An Introduction to mdlout
author: Jane Doe
---
```

The body has one '#heading per slide. The body of each slide can contain a bullet list, a paragraph or two of prose, and *not much else* on the currentslidesf-package. Seeexamples/slides_basic.md' for the documented workarounds.

18. Appendix D: Implementation notes

This appendix records design decisions that did not fit into the main flow of the manual. The decisions are presented as questions and answers because that is how they came up in practice.

18.1. Why a single-file Python script?

The temptation to split mdlout into multiple modules was strong, and several intermediate versions of the codebase did exactly that. The single-file structure won out for three reasons. First, the single-file form is easier to drop into a new project: a user can 'curlthe script directly off the repository's main branch and use it immediately. Second, debugging is easier when every function is in one place: a singlegrep' finds every caller. Third, the single file makes the regression-test harness simpler: there is exactly one Python entry point to test.

The cost of the single-file form is that the file is large – about three thousand lines – and that the order of definitions is sometimes more constrained than it would be in a multi-module codebase. We mitigate the first cost by aggressive use of internal section markers and a strict ordering: shared utilities at the top, parsers in the middle, generators at the bottom, the driver function dead last.

18.2. Why a flat list of Block dataclasses?

The internal representation of a parsed document is a flat list of 'Blockrecords, not a tree. Heading nesting is implicit in the heading-level field and is reconstructed at Lout-generation time by_generate_sectioned_body'. The flat list is easier to operate on because most transformations – inline conversion, footnote scan, bibliography injection – are local. The tree form would be more faithful to the document structure but would require every transformation to know how to walk it.

18.3. Why three Lout passes?

Lout itself is one-pass. The three-pass structure is a property of *cross-reference resolution*: when a document refers to a label that appears later in the document, the first pass discovers the label position,

the second pass propagates the position into the reference, and the third pass renders the resolved reference with its final pagination. Fewer passes leave references unresolved; more passes are wasted. The three-pass count is empirical and is documented in Lout's own user guide.

For documents without cross-references, a single pass would suffice. mdlout does not detect this case today; the optimisation would shave about 1.5 seconds off the build time of a typical small document but would require careful work to avoid breaking the cross-reference case.

18.4. Why ASCII PostScript output?

The Lout makefile is configured with `'CHAROUT=0-` ASCII-only PostScript output. This is roughly four times larger than the binary PostScript form but is reproducible across machines and survives text-only transport (e.g. patch files, email attachments, version control). The reproducibility matters for the regression suite, which compares PostScript output byte-by-byte; the size cost is absorbed by `ps2pdf`, which produces compact output regardless of the input encoding.

18.5. Why no caching?

mdlout does not cache the Lout build output between invocations. Each invocation re-parses the Markdown, re-generates the Lout source, and re-invokes Lout from scratch. The full build of the regression corpus takes ninety seconds; the lifetime cost of *not* caching is therefore about 1.5 minutes per build, which is small enough that the engineering cost of a cache would not pay for itself.

If you find yourself rebuilding the same document repeatedly during editing, use `'-watch` (which incurs the same per-build cost but absorbs the typing pauses) or `-serve` (which adds a live-reload SSE channel).

18.6. Why URW++ Nimbus fonts?

The HTML output embeds URW++ Nimbus fonts so the rendered text is metrically identical to the PostScript output. URW++ Nimbus is a free, openly-licensed approximation of the Adobe Type 1 base thirty-five fonts that PostScript printers historically came with; it is what Ghostscript itself uses for `'ps2pdf`. Embedding the fonts inline produces a self-contained HTML file at the cost of several megabytes per document. The `-no-font-embedding` flag disables the embed; the rendered text then falls back to the browser's system fonts, which usually look acceptable but may not match the PostScript glyph metrics exactly.

19. Appendix E: Compatibility matrix

mdlout is tested against the following toolchain combinations. Other combinations may work but are not part of the regression suite.

Component	Tested versions	Notes
Python	3.10, 3.11, 3.12	3.13 not yet validated
Lout	3.43 (the fork)	upstream 3.40 also works
Ghostscript	9.50, 9.55, 10.0	only <code>'ps2pdf</code> is used
Chromium	90-128	for browser tests only

ImageMagick	6.9, 7.x	for regression diffs only
Linux	Ubuntu 20.04-24.04	other distros work
macOS	12-15	Homebrew Ghostscript
Windows	not supported	use WSL
WSL	WSL2 on Win 10/11	the maintainer's setup

The maintainer's daily-driver environment is WSL2 (Ubuntu 22.04) on Windows 11, with Python 3.11 and the WSL-side Chromium for browser tests. This is also the configuration the regression suite is verified against on every push.

20. Appendix F: A worked migration case study

This appendix walks through a real migration from a LaTeX-based workflow to mdlout. The source document was a hundred-page graduate thesis in numerical analysis; the conversion took about three weeks of evening work. The notes below summarise what worked, what did not, and what would have made the migration easier.

20.1. The source document

The original thesis was written in LaTeX over the course of three years, with three distinct stylistic phases visible in the source: an early phase that used the standard `'articleclass` with no package-level customisation, a middle phase that switched to `thememoirclass` and added a half-dozen custom macros, and a late phase that introduced TikZ for figures and `andbiblatex` for the bibliography. The result, at conversion time, was about eight thousand lines of LaTeX source plus a private style file of about six hundred more.

The document had been built reliably with `'pdflatexandbiber'`, but the build was slow (about ninety seconds per pass, three passes per build) and the dependency on the LaTeX toolchain made it difficult to share the source with collaborators who used Word. The motivation for the mdlout migration was simpler than "better typography": it was *portability*. A Markdown source file can be opened in any editor, rendered in any browser, and converted to Word via Pandoc without losing the structural skeleton of the document.

20.2. What translated cleanly

The bulk of the document – prose paragraphs, section headings, body text, inline emphasis, in-text citations – translated mechanically. A short Python script (about fifty lines) walked the LaTeX source line by line, matched the `'\section{...}, \subsection{...}, \emph{...}, and \cite{...}'` macros against regexes, and emitted equivalent Markdown. The result was about ninety percent correct on the first try and required only minor manual editing to fix the remaining ten percent.

Math equations translated even more cleanly, because mdlout's `'$...$and$$...$$inline` syntax is exactly LaTeX-compatible. Every equation in the document compiled on the first try in mdlout's HTML mode (via KaTeX); the PDF mode was less seamless because Lout's `s@Eq` typesetter has its own syntax, and a small handful of complicated equations had to be hand-translated.

The bibliography translated mechanically as well. The original `'biblatex.bibfile` was processed by a one-line Python expression that emitted one[@key]: ...line per entry, in the same order as the `.bib` source. The numbering was identical between the two builds.

20.3. What did not translate

Three classes of content required hand-work:

TikZ figures. TikZ does not have an equivalent in Lout, and mdlout does not attempt to convert TikZ source. The migration script listed every TikZ block by line number; each one was hand-converted either to a raw-Lout `@Diag` block (for simple flowcharts) or to a pre-rendered SVG image (for everything else). About forty figures were converted in this way; each one took fifteen to thirty minutes.

Custom LaTeX macros. The author had defined about a dozen custom macros, each of which was a syntactic abbreviation for a specific piece of body text. The macros translated to Lout `@Defblocks` in a `custommydefs` file; the conversion was mechanical but tedious.

Margin notes. LaTeX's `\marginpar` macro has no direct equivalent in mdlout. The migration script flagged every occurrence and the author hand-rewrote each one either as a footnote (the usual fallback) or as a sidebar admonition (when the content was more substantial). About eighty margin notes were converted in this way.

20.4. The total time budget

The migration took three weeks of evening work, totalling about forty-five hours of effort. A breakdown by phase:

Phase	Hours	Notes
Initial parsing script	4	One Sunday afternoon
Manual edits after parsing	8	Mostly fixing macro edge cases
TikZ-to-SVG conversion	12	The biggest single cost
Bibliography migration	2	Surprisingly easy
Equation hand-tuning	6	Lout <code>@Eq</code> learning curve
Front-matter and templating	3	mdlout frontmatter
QA and proofreading	10	Page-by-page visual check

The largest single cost was the TikZ conversion, which is also the piece of work that mdlout could most readily reduce. A future mdlout-side TikZ importer is a clear win and is tracked as an open ticket.

20.5. Lessons learned

A few observations that might save effort on similar migrations:

- Translate prose first, figures last.** The prose translates mechanically; the figures translate one-by-one. Doing the prose first lets you proof-read the structure of the document before spending hours on individual diagrams.
- Preserve the bibliography.** The LaTeX `.bibformat` is close enough to mdlout's `s[@key] : ...` syntax that a one-line script can convert it. Do not retype.

3. **Use both output formats during the migration.** The HTML mode is much faster to iterate on (no ‘ps2pdf’ step) and most issues that surface in HTML also surface in PDF. The PDF build should be reserved for final-pass proofing.
4. **Adopt frontmatter early.** A correct frontmatter block is the difference between a document that builds cleanly and a document that produces fifty pages of plain-typewriter output with the wrong margins. Get the frontmatter right on day one.

21. References and further reading

The mdlout codebase ships with three additional manuals: the top-level ‘README.md’, the tutorial `underdocs/tutorial.md`, and the architecture `notedocs/ARCHITECTURE.md`. The Lout User’s Guide is bundled with the submodule `underlout/doc/user/`.^[6]¹

Department of Computer Science, University of Sydney, third edition,

1. The canonical reference for everything Lout-related.

22. References

1. [cite]
2. [key]

¹J. H. Kingston. *The Lout Document Formatting System*.